Agent Postmortem Generator

Part 12

Adapter Architecture & Plugin System

Purpose

Define the contract, discovery rules, and execution boundaries for framework-specific adapters so the core pipeline remains deterministic, local-first, and framework agnostic.

Project: Agent Postmortem Generator

Design Document

1. Executive Overview

The Adapter Architecture & Plugin System defines how Agent Postmortem Generator connects framework-specific trace sources to the project-owned pipeline without leaking those source details into the core model, analysis, or rendering layers.

It exists because the project must accept traces from multiple agent frameworks over time, but the rest of the system can only remain reliable if every framework-specific quirk is contained at a narrow boundary.

In the MVP, this module is intentionally small: it discovers adapters, validates adapter metadata, selects a compatible adapter deterministically, and returns a generic trace bundle that the existing parser can consume.

The module sits at the outermost edge of the ingestion path. It is upstream of parsing and validation, and it must stop before any canonical trace interpretation begins.

2. Problem Statement

Modern agent frameworks do not agree on trace structure, event naming, error semantics, or how execution state should be represented. One framework might expose structured tool-call events; another might bury them inside textual logs; a third may split one task across several nested spans. If the core pipeline learns all of those shapes directly, the system becomes brittle, hard to test, and impossible to extend without continual regressions.

The hard part is not parsing files. The hard part is creating a strict architectural boundary so that framework-specific logic can evolve independently while the rest of the project continues to operate on stable, framework-agnostic inputs.

The MVP needs this module for two reasons. First, the project already commits to framework agnosticism after normalization, which requires a boundary where framework knowledge lives. Second, even with only one generic JSON trace format in the MVP, the adapter contract must be established now so future adapters do not force a redesign of the core pipeline.

3. Goals

Goal	What success looks like
Contain framework knowledge	Framework-specific parsing lives only in adapters, not in parser, validation, normalizer, or analysis code.
Deterministic selection	Given the same inputs and environment, adapter discovery and selection produce the same result every time.
Explicit compatibility	Each adapter declares the frameworks, versions, and output schema it supports.
Stable handoff	Adapters emit a project-owned generic trace bundle and provenance metadata, not framework-native objects.
Small MVP footprint	The system remains simple enough for 2 to 3

	engineers to maintain without a plugin platform team.

- Expose a minimal adapter contract that can support both built-in and external adapters.
- Keep adapter loading local and in-process for the MVP.
- Make selection rules explicit, deterministic, and testable.
- Provide structured diagnostics when an adapter is missing, incompatible, or fails during execution.

4. Non-Goals

- No remote plugin marketplace or cloud-based adapter registry.
- No sandboxing, process isolation, or WASM-based adapter runtime in the MVP.
- No probabilistic adapter matching or model-driven framework detection.
- No direct canonical-model construction inside adapters; that belongs to the normalizer and downstream layers.
- No UI-specific adapter logic, report-generation logic, or output-format logic.
- No dynamic hot reloading of plugins while the process is running.
- No network access requirements for adapter discovery or execution.

5. Responsibilities

Owns	**Must never own**
Adapter manifest schema and versioning rules	Canonical trace model semantics
Discovery of built-in and installed adapters	Parsing generic JSON trace documents
Deterministic adapter selection	Analysis, causality reconstruction, or postmortem generation
Execution context passed into adapters	UI presentation or output formatting
Compatibility checks between source type and adapter version	Framework-specific semantics outside adapter boundaries
Adapter-level diagnostics and provenance metadata	Normalization policy beyond the adapter output contract

The core boundary rule is simple: an adapter may understand the source framework deeply, but it may only return data in the project-approved handoff format. If it needs to make a framework-specific judgment, that judgment must be expressed as trace evidence or adapter metadata, not as hidden core logic.

6. Inputs

The module accepts a source trace request, not a canonical trace. A request may be a local file, a directory, a bundle of raw framework artifacts, or a structured source descriptor passed from the CLI or UI.

- Optional explicit adapter hint, such as an adapter identifier or source type.

- One or more local input paths.
- Optional source metadata supplied by the caller, such as framework name, version, or export mode.
- Adapter configuration values that are safe to use locally and do not require remote secrets.

Validation rules:

- The requested adapter must exist and declare support for the source type or framework version being processed.
- The adapter manifest must include a stable adapter identifier, an adapter API version, and a declared output contract version.
- Input paths must exist and must be readable before adapter execution begins.
- The adapter must not claim compatibility that conflicts with the project's supported adapter API version.
- If auto-detection is used, the selection engine must produce exactly one winning adapter or fail with an explicit ambiguity error.

Assumptions:

- Adapters run in the same local Python process in the MVP.
- Input sources are owned by the user and are not modified in place.
- Adapter execution should be deterministic for the same input and configuration, aside from timestamps and other explicitly excluded volatile fields.

Malformed input handling:

- Unknown adapter hint: fail fast with a clear list of known adapter IDs.
- Unreadable source bundle: fail before adapter invocation.
- Manifest schema mismatch: reject the adapter and report the compatibility gap.
- Adapter runtime failure: surface a structured adapter error with a stack trace for local debugging, but do not crash the broader pipeline.

7. Outputs

The module emits a project-owned generic trace bundle together with metadata that describes where it came from and how it was produced.

Output	Description
Generic trace bundle	The standardized JSON trace document consumed by the parser and validation layers.
Adapter provenance	Adapter ID, adapter version, source type, input fingerprints, and execution timestamp.
Adapter diagnostics	Warnings, recoverable mismatches, and compatibility notes.
Structured errors	Machine-readable failure codes for selection, loading, or execution failures.

Human-readable output should remain brief and operational. The adapter layer should explain what it did and what it could not do, but it should not attempt to interpret the failure in narrative form.

8. Internal Design

The module is split into five subcomponents:

- Adapter Manifest: a declarative description of identity, supported source types, output contract version, and compatibility rules.
- Discovery Loader: finds built-in adapters and installed external adapters from the local Python environment.
- Selection Engine: resolves the adapter to use based on explicit hints, declared capabilities, and deterministic tie-break rules.
- Execution Host: invokes the selected adapter in-process with a read-only execution context and captures its output.
- Contract Validator: checks the returned generic trace bundle and metadata before handing the result to the parser.

Workflow:

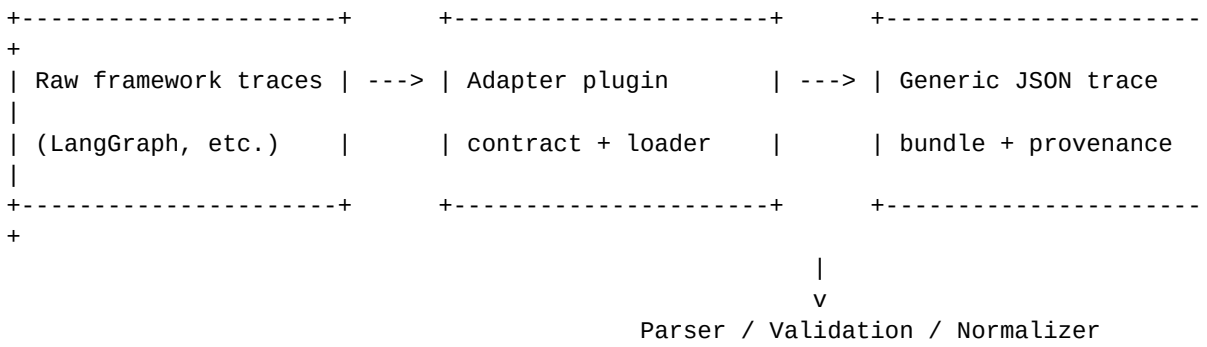
```
1. Collect available adapter manifests from built-in packages and installed plugin
entry points.
2. Filter candidates using explicit caller hints, source type declarations, and
supported framework versions.
3. Apply deterministic ranking rules if more than one candidate remains.
4. Load the selected adapter and build an immutable execution context.
5. Execute the adapter locally.
6. Validate the emitted generic trace bundle and provenance metadata.
7. Return the bundle to the parser, or return a structured adapter error if
validation fails.
```

Data flow:

```
Source trace files / raw framework artifacts
        -> Adapter discovery and selection
        -> Adapter execution
        -> Generic JSON trace bundle + provenance
        -> Parser / validation / normalization pipeline
```

Important internal decisions:

- Adapters are loaded in-process for the MVP to avoid subprocess orchestration complexity.
- The loader must not silently prefer one adapter over another when compatibility is ambiguous.
- Adapter execution is treated as pure transformation over local inputs; no adapter should be required to talk to a remote service.
- The adapter layer should never infer the canonical model directly. It only emits the project-approved source bundle.



9. Data Model

The adapter system uses a small set of explicit, versioned data objects.

Entity	Key fields	Why it exists
AdapterManifest	adapter_id, name, version, api_version, source_types, output_contract_version, priority, entrypoint, deterministic	Declares what the adapter is and what it supports.
AdapterInvocationContext	source_paths, source_type, explicit_adapter_id, framework_version, working_directory, env_policy, config	Provides the adapter with only the information it needs to transform a source bundle locally.
AdapterResult	trace_bundle, provenance, warnings, emitted_artifacts	Packages the standardized output and the evidence of how it was produced.
AdapterError	code, adapter_id, message, details, recoverable, hint	Makes failures machine-readable and actionable.
SourceFingerprint	file extension, magic bytes, root keys, embedded framework markers	Supports deterministic auto-detection when the caller does not specify an adapter.

Relationship rule: one adapter manifest may support multiple source types or framework versions, but one adapter invocation produces exactly one generic trace bundle. The bundle is intentionally not the canonical trace model; it is the standardized input artifact consumed downstream.

10. Algorithms / Processing Logic

Adapter discovery and selection must follow a deterministic, explainable order:

1. Prefer an explicit adapter ID provided by the caller.
2. If no explicit adapter is provided, filter by source type and any declared framework markers.
3. Discard any adapter whose manifest API version is incompatible with the current host API version.
4. Rank remaining adapters by declared priority, then by specificity of source support, then by stable adapter ID ordering.
5. If more than one adapter remains with equal rank, fail with an ambiguity error rather than guessing.

Execution rules:

6. Build a read-only execution context from the caller input and source metadata.
7. Invoke the adapter locally and capture only the standardized output contract.
8. Strip or flag any adapter output that does not belong to the project-approved generic trace bundle.
9. Reject the result if the bundle schema is invalid or if mandatory provenance fields are missing.
10. Pass the validated bundle to the parser without interpreting framework semantics in the adapter layer.

Edge cases and decision rules:

- If a bundle contains multiple trace candidates, the adapter must choose one deterministically or emit an error explaining why selection was impossible.
- If two installed adapters claim the same source type and equal priority, the module must not arbitrarily pick one.
- If an adapter returns warnings alongside a valid bundle, the warnings should be attached to provenance and surfaced to the caller, but they must not change the bundle's semantics.
- If the adapter needs to collapse framework-specific concepts into a generic event, the adapter must record the loss of specificity in its diagnostics rather than hiding it.

Deterministic behavior requirements:

- Discovery order must be stable.
- Selection ranking must be stable.
- Validation must not depend on process timing or hash randomization.
- Any non-deterministic adapter behavior is a defect, not a supported feature.

11. Tradeoffs

Tradeoff	Chosen approach	Why
Simplicity vs flexibility	Simple in-process plugin loading with explicit manifests	Keeps the MVP maintainable while preserving room for future adapters.
Generality vs specificity	A general adapter contract with narrow supported output	Supports multiple frameworks without widening the core pipeline.
Determinism vs heuristic inference	Deterministic selection and explicit failure on ambiguity	Trustworthy behavior matters more than convenience here.
Builtin registry vs open discovery	Builtin registry plus optional local entry-point discovery	Keeps first-party adapters easy to ship while supporting future external plugins.

The preferred design is intentionally less clever than a fully dynamic plugin platform. That is the correct tradeoff for an MVP whose primary value is stable, evidence-based debugging. The module should make extension possible, not indulgently easy.

12. Risks and Failure Modes

- Ambiguous adapter matches can produce incorrect or inconsistent imports if the selection rules are not strict enough.
- Adapters can accidentally leak framework-specific assumptions into the generic trace bundle if the output contract is underspecified.
- In-process plugins can misbehave or crash the host process if the module does not isolate error reporting carefully.
- A broad plugin API can become a maintenance burden if the first release tries to support too many source shapes at once.
- Auto-detection based on file naming alone is fragile and should never be the only signal.

- Adapters may silently drop information during framework-to-generic translation; this must be surfaced as diagnostics, not hidden.

The hidden complexity is version drift. Framework APIs evolve quickly, and adapter compatibility will fail more often than the rest of the system. The module therefore needs versioned manifests, explicit compatibility checks, and strong regression tests against representative framework exports.

13. Testing Strategy

Test type	What to verify
Unit tests	Manifest parsing, compatibility checks, ranking order, fingerprint matching, and error taxonomy.
Integration tests	Built-in adapter discovery and successful round-trip from sample source artifacts to generic trace bundles.
Fixture-driven tests	Representative framework exports across versions, malformed bundles, ambiguous matches, and adapter warnings.
Regression protection	A fixed corpus of source traces that must continue to resolve to the same adapter and output contract.

- Test that explicit adapter selection always overrides auto-detection when compatible.
- Test that ambiguous candidates fail loudly instead of falling back to a guess.
- Test that adapter output schema validation rejects missing provenance fields.
- Test that sample framework traces remain reproducible across dependency upgrades.
- Test that adapter errors are structured enough for the CLI and UI to present useful guidance.

14. Implementation Checklist

11. Define the adapter manifest schema and version the adapter API.
12. Implement the discovery loader for built-in adapters and local entry points.
13. Implement deterministic selection rules and ambiguity failures.
14. Add the execution context object and a structured result/error contract.
15. Implement the built-in adapter for the MVP-supported generic JSON path or first source type.
16. Add schema validation for adapter outputs and provenance metadata.
17. Create adapter fixture corpora for each supported source type and framework version.
18. Wire adapter errors into CLI and UI friendly diagnostics.
19. Document compatibility policy, versioning expectations, and extension rules.
20. Lock the module down with regression tests before adding any new framework support.

A useful implementation discipline is to treat every new adapter like a protocol change. If the contract needs to change, the adapter API version changes first, and compatibility is renegotiated explicitly.

15. Future Expansion

- External adapter packages discovered via standard Python entry points with richer manifest metadata.
- Optional subprocess or sandboxed adapter execution for untrusted third-party plugins.
- Capability negotiation beyond source type, such as partial support for specialized event families.
- Adapter-generated enrichment metadata, kept separate from the generic trace bundle.
- A lightweight compatibility report that explains why one adapter was chosen over another.
- A future plugin catalog or installation helper, if the ecosystem grows large enough to justify it.

These expansions should only be added if the project actually accumulates a meaningful adapter ecosystem. Premature plugin-platform work would dilute the core product and add maintenance debt without improving the debugging experience.

16. Final Recommendation

This design is MVP-ready if it stays small, in-process, and contract-first. The module should ship with built-in adapter loading, deterministic selection, and strong schema validation, but it should avoid sandboxing, marketplace mechanics, and other plugin-platform extras.

The main simplification still recommended is to keep the first implementation focused on one or two well-understood built-in adapters and a narrow manifest schema. That keeps the boundary real without turning the adapter layer into a project of its own.

Document Notes

- This document intentionally defines only the adapter boundary and plugin system. Parsing, validation, normalization, analysis, report generation, and output rendering remain separate modules.
- Every adapter must preserve the project principle that evidence is the source of truth. If an adapter cannot faithfully represent a framework event, it must say so explicitly.
- The adapter layer is a boundary, not a place to hide complexity.